ManagedTorrent 상태 머신 분석

개요

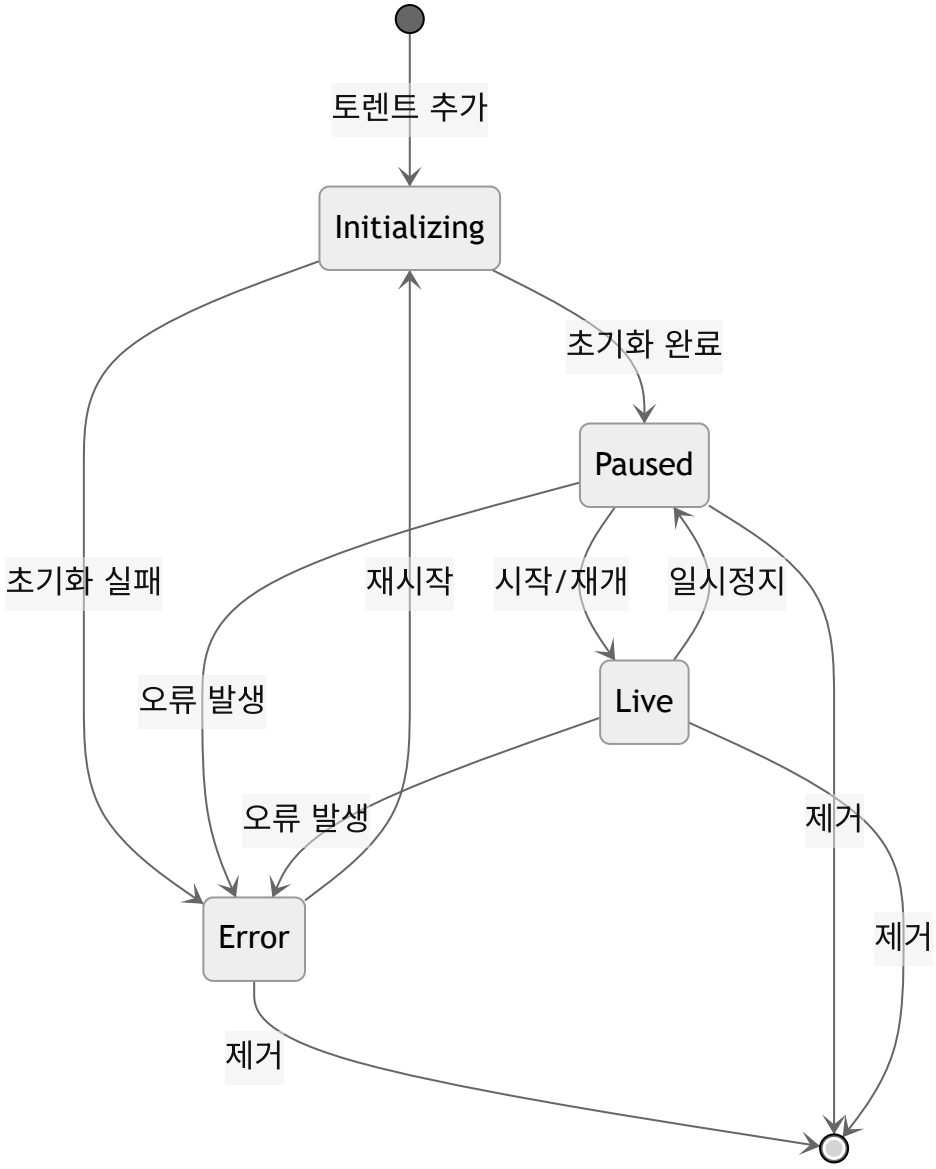
이 문서는 rqbit BitTorrent 클라이언트의 `ManagedTorrent` 상태 머신을 상세히 분석합니다. ManagedTorrent는 개별 토렌트의 생명주기를 관리하며, 초기화부터 완료까지의 모든 상태 전이를 담당합니다.

ManagedTorrentState 열거형

`ManagedTorrentState` 는 토렌트의 현재 상태를 나타내는 열거형으로, 다음과 같이 정의됩니다:

```
pub enum ManagedTorrentState {
Initializing(Arc<TorrentStateInitializing>),
Paused(TorrentStatePaused),
Live(Arc<TorrentStateLive>),
Error(anyhow::Error),
None, // 상태 전환 중에만 사용되는 임시 상태
}
```

상태 전이 다이어그램



각 상태별 상세 분석

1. Initializing 상태

목적 및 역할

- **파일 무결성 검사**: 기존 파일들의 해시 검증
- **메타데이터 처리**: 토렌트 정보 파싱 및 검증
- **스토리지 준비**: 파일 시스템 구조 생성
- **FastResume 처리**: 이전 세션 상태 복원

핵심 구조체

```rust
pub struct TorrentStateInitializing {
pub(crate) files: FileStorage,
pub(crate) shared: Arc<ManagedTorrentShared>,
pub(crate) metadata: Arc<TorrentMetadata>,
pub(crate) only_files: Option<Vec<usize>>,
pub(crate) checked_bytes: AtomicU64,
previously_errored: bool,
}
```

주요 필드 분석

- **files**: 파일 스토리지 인터페이스
- **shared**: 토렌트 공통 정보 (ID, 해시, 설정 등)
- **metadata**: 토렌트 메타데이터 (파일 정보, 조각 정보 등)
- **only_files**: 선택적 다운로드 파일 목록
- **checked_bytes**: 검증된 바이트 수 (진행률 표시용)
- **previously_errored**: 이전 오류 상태에서 재시작 여부

초기화 과정 (`check` 메서드)

1. **FastResume 데이터 로드**

```rust
let have_pieces = bitv_factory
.load(id)
.await
```

```
.context("error loading have_pieces")?;
```

2. **FastResume 검증**

```
let have_pieces = self.validate_fastresume(&*bitv_factory, have_pieces).await;
```

- 비트필드 길이 검증
- 무작위 조각 샘플링을 통한 무결성 확인
- 각 파일당 최소 1개 조각 검증

3. **전체 무결성 검사** (FastResume 실패 시)

```
let have_pieces = FileOps::new(&self.metadata.info, &self.files, &self.metadata.
.initial_check(&self.checked_bytes);
```

4. **ChunkTracker 생성**

```
let chunk_tracker = ChunkTracker::new(
have_pieces.into_dyn(),
selected_pieces,
*self.metadata.lengths(),
&self.metadata.file_infos,
);
```

5. **파일 길이 설정**

```
self.files.ensure_file_length(idx, fi.len)
```

2. Paused 상태

목적 및 역할

- **대기 상태**: 다운로드 준비 완료, 실행 대기
- **상태 보존**: 진행 상황 및 메타데이터 유지
- **설정 변경**: only_files 등 설정 수정 가능

핵심 구조체

```rust
pub struct TorrentStatePaused {
pub(crate) shared: Arc<ManagedTorrentShared>,
pub(crate) metadata: Arc<TorrentMetadata>,
pub(crate) files: FileStorage,
pub(crate) chunk_tracker: ChunkTracker,
pub(crate) streams: Arc<TorrentStreams>,
}
```

주요 기능

- **진행 상황 조회**: `hns()` 메서드를 통한 Have/Need/Selected 정보
- **파일 선택 업데이트**: `update_only_files()` 메서드
- **스트림 관리**: 파일 스트리밍을 위한 준비

3. Live 상태

목적 및 역할

- **활성 다운로드**: 피어 연결 및 데이터 교환
- **피어 관리**: 연결, 요청, 응답 처리
- **진행률 추적**: 실시간 다운로드 상태 모니터링
- **업로드 처리**: 다른 피어에게 데이터 제공

아키텍처 개요

Live 상태는 복잡한 멀티태스킹 시스템으로 구성됩니다:

1. **공유 잠금 상태**

- `TorrentStateLocked` : 조각 상태의 단일 진실 소스
- 피어 상태: DashMap을 통한 샤딩된 해시맵

2. **태스크 (액터들)**

- **피어 추가 태스크**: 새로운 피어 연결 관리
- **피어 관리 태스크**: 개별 피어와의 통신
- **청크 요청 태스크**: 피어별 조각 요청 관리

피어 생명주기

```
queued (초기) -> connected (연결됨) -> live (활성)
↓ ↓ ↓
dead <---------- dead <-------- dead
↓ ↓ ↓
not_needed <-- not_needed <-- not_needed
```

주요 기능

- **피어 연결 관리**: TCP/uTP 연결 처리

- **메시지 교환**: BitTorrent 프로토콜 메시지 처리
- **조각 요청**: 효율적인 다운로드 스케줄링
- **업로드 관리**: choke/unchoke 알고리즘
- **확장 프로토콜**: ut_metadata, ut_pex 등

4. Error 상태

목적 및 역할

- **오류 보고**: 발생한 오류 정보 저장
- **복구 준비**: 재시작 가능한 상태 유지
- **리소스 정리**: 오류 발생 시 안전한 정리

오류 처리 메커니즘

```
fn stop_with_error(&self, error: anyhow::Error) {
let mut g = self.locked.write();
match g.state.take() {
ManagedTorrentState::Live(live) => {
if let Err(err) = live.pause() {
warn!("error pausing live torrent during fatal error handling: {err:#}");
}
}
// ... 기타 상태 처리
};
g.state = ManagedTorrentState::Error(error)
```

```
}
```

상태 전이 메커니즘

1. 상태 전이 함수들

`start` 메서드

```
pub(crate) fn start(
self: &Arc<Self>,
peer_rx: Option<PeerStream>,
start_paused: bool,
) -> anyhow::Result<()>
```

- **Initializing → Paused**: 초기화 완료 후
- **Paused → Live**: 시작 명령 시
- **Error → Initializing**: 재시작 시

`pause` 메서드

```
pub(crate) fn pause(&self) -> anyhow::Result<()>
```

- **Live → Paused**: 일시정지 명령 시
- 활성 연결 정리 및 상태 저장

2. 상태 변경 알림

알림 메커니즘

```
pub(crate) state_change_notify: Notify,
```

- 상태 변경 시 대기 중인 태스크들에게 알림
- `wait_until_initialized()`, `wait_until_completed()` 등에서 사용

3. 동시성 제어

잠금 구조

```
pub(crate) locked: RwLock<ManagedTorrentLocked>,
```

```
pub(crate) struct ManagedTorrentLocked {
pub(crate) paused: bool,
pub(crate) state: ManagedTorrentState,
pub(crate) only_files: Option<Vec<usize>>,
}
```

상태별 부작용 및 정리 작업

1. Initializing → Paused

- **부작용**: 파일 시스템 구조 생성, 비트필드 저장
- **정리**: 임시 검증 데이터 해제

2. Paused → Live

- **부작용**:

- 피어 연결 태스크 시작
- 트래커 announce 시작
- DHT 피어 검색 시작
- **정리**: 없음 (상태 유지)

3. Live → Paused

- **부작용**:
- 모든 피어 연결 종료
- 진행 상황 저장
- 백그라운드 태스크 정리
- **정리**: 네트워크 리소스 해제

4. Any → Error

- **부작용**:
- 오류 로깅
- 상태 알림
- **정리**:
- Live 상태인 경우 pause() 호출
- 리소스 안전 해제

성능 최적화 기법

---

1. 상태 전환 최적화

- **원자적 전환**: `take()` 메서드를 통한 안전한 상태 교체

- **최소 잠금**: 필요한 순간에만 쓰기 락 획득

2. 메모리 효율성

- **Arc 공유**: 메타데이터와 공유 정보의 참조 카운팅
- **지연 초기화**: 필요시에만 리소스 할당

3. 동시성 최적화

- **세마포어**: 동시 초기화 제한
- **비동기 처리**: 모든 I/O 작업 비동기화

오류 복구 전략

1. 자동 재시작

```
Error → Initializing: 사용자 명령 또는 자동 재시도
```

2. 상태 보존

- 오류 발생 시에도 메타데이터 유지
- FastResume 데이터 보존

3. 점진적 복구

- 부분적 데이터 손실 시 해당 조각만 재다운로드
- 전체 재검증 회피

결론

ManagedTorrent 상태 머신은 다음과 같은 특징을 가집니다:

1. **명확한 상태 분리**: 각 상태의 책임과 역할이 명확히 정의됨
2. **안전한 전이**: Rust의 타입 시스템을 활용한 안전한 상태 전환
3. **효율적인 리소스 관리**: 상태별 적절한 리소스 할당 및 해제
4. **강력한 오류 처리**: 다양한 오류 상황에 대한 복구 메커니즘
5. **확장 가능한 설계**: 새로운 상태나 기능 추가 용이

이러한 설계를 통해 각 토렌트의 생명주기를 안정적이고 효율적으로 관리할 수 있습니다.